

Шпаргалка. Spring Data JPA

Введение

Вы уже умеете создавать веб-приложения, которые работают по паттерну **Controller-Service-Repository**. Они используют класс, который имитирует работу репозитория (хранилища данных) с помощью внутренней коллекции, например список или множество объектов. В реальных приложениях вам придется использовать настоящие полноценные базы данных.

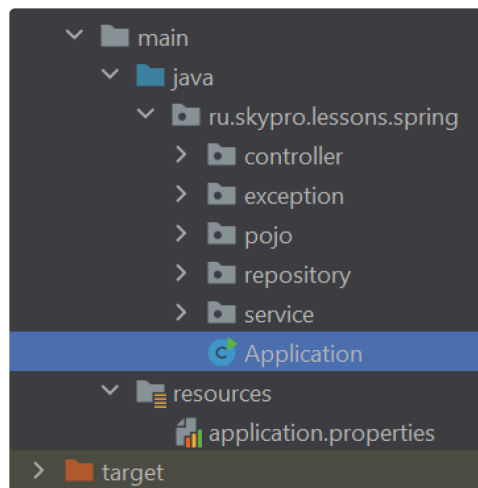
В этом уроке вы познакомитесь со **Spring Data JPA** — мощным инструментом, который позволяет легко интегрировать ваши приложения с базами данных, используя Java Persistence API (JPA).

⚠ Модуль **Spring Data JPA** является частью семейства Spring Data. **Spring Data** упрощает создание приложений на основе **Spring**, предоставляя расширенную поддержку для работы с реляционными базами данных.

Spring Data JPA содержит в себе уже знакомый нам фреймворк **Hibernate**. Модуль добавляет к нему дополнительные возможности для создания и управления классами-репозиториями.

Подключение Spring-Boot-приложения к БД

Чтобы подключить базу данных к приложению, необходимо указать, где находится база данных. Для этого используется файл со свойствами приложения. Располагается он в директории `resources`.



В нем необходимо указать следующие параметры:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/postgres
spring.datasource.username=postgres
spring.datasource.password=postgres
# Hibernate ddl auto (create, create-drop, validate, update)
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Какие значения может принимать `spring.jpa.hibernate.ddl-auto` ?

- `none` — отсутствует инициализация.
- `create` — при запуске приложения удаляет все созданные таблицы с данными и создает их заново.
- `create-drop` — создает схему при запуске и уничтожает схему при закрытии контекста. Полезно для модульных тестов.
- `validate` — проверяет, соответствуют ли сущности базы объектам в Java. Если не совпадает, запуск приложения невозможен. Не вносит изменений в базу данных.
- `update` — обновляет схему только при необходимости. Например, если в объект было добавлено новое поле, то оно просто изменит таблицу для нового столбца, не уничтожая данные.

⚠ Рекомендуется использовать `update` — так вы не потеряете данные, а при запуске приложения все новые изменения в коде применятся к базе.

Также необходимо подключить:

- стартер для работы с Spring JPA;
- драйвер для работы с PostgreSQL.

Для этого нужно добавить в `dependencies` еще один кусочек кода:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
  <version>3.0.6</version>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.3.2</version>
</dependency>
```

Теперь приложение готово к работе.

Однако для использования Hibernate мы должны создать **Entity**. Давайте приведем наш ро-о-объект к требованиям Entity:

```
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;
    private int salary;

    public Employee() {
    }

    public Employee(Integer id, String name, int salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getSalary() {
        return salary;
    }

    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```

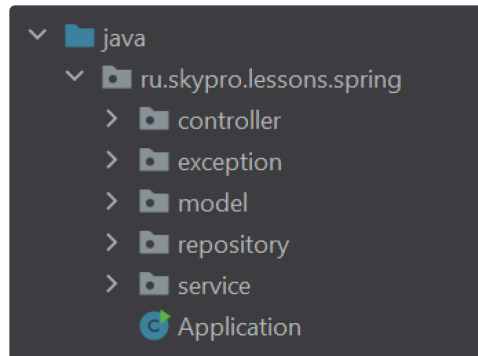
CrudRepository

Для взаимодействия с БД необходимы репозитории.

★ **Репозитории** — это интерфейсы, которые предоставляют готовые методы для выполнения стандартных операций, таких как сохранение, удаление, поиск и обновление объектов в базе данных.

Ранее мы писали SQL-запросы вручную. Благодаря Spring репозитории сами формируют запросы. Мы, как разработчики, лишь описываем в Java-коде действия, которые нам нужны.

Репозитории лежат в отдельном пакете `repository`.



Чтобы создать репозиторий, нужно выполнить следующие шаги:

1. Создайте интерфейс, который будет нашим репозиторием.
2. Унаследуйте этот интерфейс от одного из предоставляемых Spring Data JPA интерфейсов, таких как `JpaRepository`, `CrudRepository` или `PagingAndSortingRepository`. В качестве параметров типа укажите вашу сущность и тип ее первичного ключа.
3. Определите в интерфейсе дополнительные методы, если требуется выполнить специфические запросы к базе данных. Spring Data JPA автоматически сгенерирует запросы на основе названий этих методов.

Давайте подробнее разберем интерфейсы, которые наследуют реализацию от `CrudRepository`.

```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {  
  
}
```

Для создания интерфейса `CrudRepository` необходимо указать класс, с которым нужно работать, и тип его идентификатора. В нашем примере это `Employee` и его идентификатор `Integer`.

У них уже есть ряд методов, которые доступны нам. Вот несколько примеров:

- `save()` — сохраняет изменения в БД;
- `findOne()` — находит первый элемент, который удовлетворяет определенным условиям;
- `findById()` — используется, если необходимо совершить поиск по идентификатору;
- `findAll()` — возвращает все записи из таблицы;
- `count()` — возвращает число long — количество записей в таблице;
- `deleteById()` — удаляет сущность из БД по ее идентификатору.

Использование репозиторий в сервисах

Модели настроены, репозитории созданы и подключены к БД.

Чтобы начать их использовать, необходимо сначала внедрить нужные репозитории в сервис. Сделать это можно через конструктор:

```
@Service  
public class EmployeeServiceImpl implements EmployeeService {  
  
    // Создаем переменную для доступа к репозиторию сотрудников  
    private final EmployeeRepository employeeRepository;  
  
}
```

```
// Создаем конструктор, который принимает репозиторий сотрудников
// Так мы внедрим зависимость репозитория сотрудников в данный сервис
public EmployeeServiceImpl(EmployeeRepository employeeRepository) {
    this.employeeRepository = employeeRepository;
}
// Здесь будут другие методы для работы с сотрудниками
...
}
```

Какие методы можно использовать?

Метод создания сущности

Для создания сущности у `EmployeeRepository` есть метод `save()`. Он ожидает получение модели, чтобы сохранить ее.

```

@Override
public void addEmployee(Employee employee) {
    employeeRepository.save(employee);
}

```

Всего лишь одной командой мы сохранили данные модели в БД.

Метод `save()` можно использовать для создания новых записей.

Метод получения данных модели

Если мы хотим получить данные модели по определенному `id`, то можно использовать метод `findById()`, который возвращает `Optional`.

```

// Реализация метода getEmployeeById интерфейса EmployeeService
// Метод возвращает сотрудника по указанному id
@Override
public Employee getEmployeeById(int id) {
    // Используем метод findById() репозитория для получения сотрудника по id
    // Возвращает Optional<Employee>, который может содержать сотрудника или быть пустым
    Optional<Employee> employeeOptional = employeeRepository.findById(id);
    // Если сотрудник найден, возвращаем его
    // Иначе выбрасываем исключение с указанием некорректного id
    return employeeOptional.orElseThrow(() -> new IncorrectEmployeeIdException(id));
}

```

Метод удаления

Для удаления существует метод `deleteById()`. Он принимает идентификатор сущности, которую необходимо удалить из БД. Этот метод ничего не возвращает.

```

@Service
public class EmployeeServiceImpl implements EmployeeService {

    // Создаем переменную для доступа к репозиторию сотрудников
    private final EmployeeRepository employeeRepository;

    // Создаем конструктор, который принимает репозиторий сотрудников
    // Это нужно, чтобы внедрить зависимость репозитория сотрудников в данный сервис
    public EmployeeServiceImpl(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    // Метод для получения сотрудника по идентификатору
    @Override
    public Employee getEmployeeById(int id) {
        return employeeRepository.findById(id)
            .orElseThrow(() -> new IncorrectEmployeeIdException(id));
    }

    // Метод для удаления сотрудника по идентификатору
    @Override
    public void deleteEmployeeById(int id) {
        employeeRepository.deleteById(id);
    }
}

```

```
// Метод для добавления нового сотрудника
@Override
public void addEmployee(Employee employee) {
    employeeRepository.save(employee);
}
}
```

Мы рассмотрели, как в сервисах взаимодействовать с репозиториями, чтобы сохранять данные в БД. Но что, если нам надо получить сотрудника с определенным значением в поле `name`? Что нам делать, если, например, мы хотим получить все позиции с `name = "Василий"`?

Составление своих запросов

Чтобы Spring начал искать сущности по определенным полям, необходимо нужным образом сформулировать имя метода в репозитории. Например, мы хотим получить все объекты

`Employee`, у которых поле `name = "Василий"`.

Создадим метод в репозитории:

```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {
    List<Employee> findByName(String name);
}
```

Далее начнем использовать его в сервисе:

```
@Override
public Employee getEmployeeByName(String name) {
    return employeeRepository.findByName(name).get(0);
}
```

А если мы захотим искать по любому другому полю, то достаточно в названии метода `findByName` заменить `Name` на название другого поля.

Также вы можете искать по нескольким полям, используя логические операторы:

`AND` или `OR`. Например, поиск по идентификатору и имени будет выглядеть так:

```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {
    List<Employee> findByNameAndId(String position, Integer id);
}
```

А если мы захотим найти позицию, у которой в поле `Name` присутствует слово `Developer`, то нам следует использовать методы со словом `Like` в конце. Например:

```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {
    List<Employee> findByNameLike(String position);
}
```

Существует большое разнообразие доступных операций. Ниже приведены основные из них.

GreaterThan

`findByIdGreaterThan(int age)` — эта операция выполняет поиск данных, у которых значения в поле больше указанного.

LessThan

`findByIdLessThan(int age)` — эта операция производит поиск данных, у которых значения в поле меньше указанного.

Between

`findByIdBetween(int from, int to)` — эта операция производит поиск данных, у которых значения поля находятся между двух указанных значений.

IsNotNull, NotNull

`findByNameNotNull()` — с помощью этой операции можно произвести поиск значений, у которых определенное поле непустое.

IsNull, Null

`findByNameNull()` — операция для поиска значений, у которых определенное поле пустое.

Like

`findByNameLike(String name)` — операция, выполняющая поиск по совпадению.

@Query

Чтобы реализовать более сложные запросы в репозитории, мы можем использовать аннотацию `@Query` и аннотировать ею метод. Ее атрибут `value` содержит JPQL или SQL для выполнения.

По умолчанию аннотация ожидает на вход JPQL-запрос:

```
@Query("SELECT e FROM Employee e")
List<Employee> findAllEmployees();
```

Если мы хотим использовать нативный SQL, то нам нужно дополнительно указать в аннотации параметр `nativeQuery= true`:

```
@Query(value = "SELECT * FROM employee",
nativeQuery = true)
List<Employee> findAllEmployees();
```

Некоторые методы репозитория подразумевают работу с входными параметрами. Есть два способа передачи параметров в запросе аннотации `@Query`.

1. Пронумерованные параметры.

Метод `getEmployeesByPositionId` выполнит запрос, указанный в аннотации, и подставит параметр, исходя из указанного в запросе номера и очередности в сигнатуре метода.

```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {

    @Query(value = "SELECT * FROM employee WHERE name = ?1",
nativeQuery = true)
    List<Employee> getEmployeesByName(String name);

}
```

2. Именованный параметр.

Метод `getEmployeesByPositionId` выполнит запрос, указанный в аннотации, и подставит параметр, исходя из указанного в запросе имени параметра и значения в аннотации `@Param`.

```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {

    @Query(value = "SELECT * FROM employee WHERE name= :name",
nativeQuery = true)
    List<Employee> getEmployeesByPositionId(@Param("name") String name);

}
```

PagingAndSortingRepository

Разбиение на страницы (пагинация) полезна, когда мы хотим предоставить пользователю большой объем данных небольшими порциями. Чтобы использовать пагинацию, **Spring Data** вводит интерфейс `PagingAndSortingRepository`.

После того как мы имплементируем репозиторий от `PagingAndSortingRepository`, нам необходимо:

- Создать объект `PageRequest`, который является реализацией интерфейса `Pageable`.
- Передать объект `PageRequest` в качестве аргумента методу репозитория, который мы собираемся использовать.

Вот так будет выглядеть метод сервиса, который обращается к методу `PagingAndSortingRepository.findAll(Pageable)`:

```
@Service
public class EmployeeServiceImpl implements EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeServiceImpl(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    @Override
    public List<Employee> getEmployeeWithPaging(int pageIndex, int unitPerPage) {
        Pageable employeeOfConcretePage = PageRequest.of(pageIndex, unitPerPage);
        Page<Employee> page = employeeRepository.findAll(employeeOfConcretePage);

        return page.stream()
            .toList();
    }
}
```

Мы используем статический конструктор класса `PageRequest`, в который передаем порядковый номер интересующей нас страницы (отсчет страниц начинается с нуля) и размер каждой страницы. Метод `findAll` возвращает объект `Page`, который мы с помощью стрима преобразуем в коллекцию.

Паттерн DTO

❗ При изучении Spring Data JPA нельзя обойти стороной понятие DTO. Оно играет важную роль в обработке и передаче данных, полученных из базы данных.

🌟 **DTO (Data Transfer Object)** — это распространенный паттерн передачи данных между различными слоями приложения, например между слоем сервисов и слоем пользовательского интерфейса.

Основная цель DTO — обеспечить оптимизацию и упрощение передачи данных. DTO группирует связанные атрибуты в одном объекте и избегает передачи лишней информации.

DTO широко используется:

- для сокрытия ненужных данных из сущности;
- уменьшения количества запросов к БД, когда нужно соединить две сущности в одном ответе, например получить книгу и имя автора.

⚠ Хорошей практикой является сокрытие всех наших сущностей с помощью DTO. Конечно, это способствует разрастанию кодовой базы и иногда вызывает дублирование, но взамен позволяет вносить изменения в сущности нашего приложения.

Применение паттерна в Spring-Boot-приложениях

Если классы сущностей утекают в другие слои приложения, например в контроллеры, это плохо. Работа с данными «размазывается» на классы, обрабатывающие веб-запросы. Как следствие — нарушается принцип единственной ответственности.

⚠ **Принцип единственной ответственности (Single Responsibility Principle)** гласит, что каждый класс должен иметь только одну свою ответственность. Все его методы и поля должны быть тесно связаны с этой ответственностью.

Другими словами, класс делает только одну вещь, но делает ее хорошо.

Следование данному принципу позволяет:

- создавать гибкие, стойкие к изменениям приложения;
- избегать ошибок при разработке и делать код удобным и легкочитаемым.

⚠ Представьте: есть приложение, которое обрабатывает заказы. Но в нем нет одного класса, который проводит все операции. Вместо этого для каждой операции — обработка заказа, распределение заказов по курьерам, обработка платежа и т. д. — созданы отдельные классы. И каждый класс занимается только своей работой.

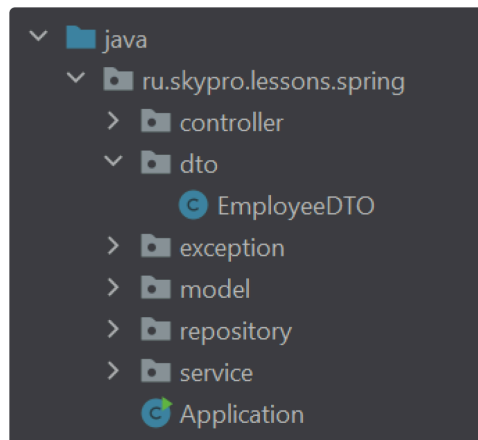
Поэтому в классических трехзвенных веб-приложениях, вместо того чтобы работать в контроллере напрямую с сущностью, чаще работают с объектами DTO.

Структуру можно описать следующим образом:

- в методах контроллера принимаются и возвращаются только DTO-объекты;
- методы сервиса также принимают и возвращают только DTO-объекты;
- внутри сервиса работа идет как с сущностями, так и с DTO-объектами.

⚠ По сути, сервис является связующим звеном между контроллером и репозиторием и содержит логику трансформации сущности в DTO и в обратную сторону.

DTO обычно размещают в отдельном package.



```
public class EmployeeDTO {

    // Поля для хранения идентификатора, имени и зарплаты сотрудника
    private Integer id;
    private String name;
    private Integer salary;

    // Метод для преобразования сущности Employee в объект EmployeeDTO
    public static EmployeeDTO fromEmployee(Employee employee) {
        EmployeeDTO employeeDTO = new EmployeeDTO();
        employeeDTO.setId(employee.getId());
        employeeDTO.setName(employee.getName());
        employeeDTO.setSalary(employee.getSalary());
        return employeeDTO;
    }

    // Метод для преобразования объекта EmployeeDTO в сущность Employee
    public Employee toEmployee() {
        Employee employee = new Employee();
        employee.setId(this.getId());
        employee.setName(this.getName());
        employee.setSalary(this.getSalary());
        return employee;
    }

    // Геттер и сеттер для идентификатора сотрудника
    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }
}
```



```

// Геттер и сеттер для имени сотрудника
public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

// Геттер и сеттер для зарплаты сотрудника
public int getSalary() {
    return salary;
}

public void setSalary(int salary) {
    this.salary = salary;
}
}

```

Как видно из примера выше, `EmployeeDTO` содержит те же поля, что и `Employee`, и не содержит никакой бизнес-логики. Кроме того, мы создали два дополнительных метода: `fromEmployee()` и `toEmployee()`, чтобы мы могли преобразовать объект модели в DTO и наоборот.

Теперь, чтобы получить сотрудников из базы данных и передать его в слой контроллеров, мы можем использовать следующий код:

```

@Service
public class EmployeeServiceImpl implements EmployeeService {

    // Инициализируем репозиторий сотрудников
    private final EmployeeRepository employeeRepository;

    // Создаем конструктор для внедрения зависимостей
    public EmployeeServiceImpl(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    // Реализуем метод получения списка всех сотрудников
    @Override
    public List<EmployeeDTO> getAllEmployees() {

        // Получаем список сотрудников из репозитория,
        // преобразуем их в DTO и собираем в список
        return employeeRepository.findAllEmployees().stream()
            .map(EmployeeDTO::fromEmployee)
            .collect(Collectors.toList());
    }
}

```

Здесь мы получаем список сотрудников из базы данных с помощью реализованного ранее метода `findAllEmployees()` репозитория, который возвращает список объектов `Employee`. С помощью стрима и вызова в нем метода `fromEmployee()` мы преобразуем каждый объект `Employee` в объекты `EmployeeDTO` и возвращаем их единой коллекцией.

Такая работа с DTO позволяет нам:

- оставить работу с сущностями в сервисе;
- иметь два независимых класса.

⚠ Если мы захотим в сущность добавить новое поле, например которое содержит дату последнего обновления сущности, нужную только на слое репозитория, то DTO и маппинг останутся неизменными. Следовательно, поведение контроллера тоже не изменится.

Этот принцип позволяет изолировать слой REST API от изменений в сущностях — и наоборот. Таким образом, мы получаем больше контроля за изменениями в разных слоях нашего приложения.

Projections

Сейчас при получении значений из базы данных репозиторий возвращает один или несколько экземпляров Entity. Только чаще всего нам нужны не все свойства возвращаемых объектов.

⚠ Чтобы возвращать себе только часть свойств, пригодятся проекции, или **Projections**.

На данный момент в нашей сущности `Employee` есть три поля: `id`, `name` и `salary`.

```
@Entity
@Table(name = "employee")
public class Employee {
    // Идентификатор сотрудника, генерируется автоматически
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    // Имя сотрудника
    private String name;
    // Зарплата сотрудника
    private Integer salary;
    // Отношение между сотрудником и его должностью (один к одному)
    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "position_id")
    private Position position;
    // Геттеры, сеттеры, конструкторы будут добавлены ниже
}
```

Допустим, нам нужно получить только поля `name` и `salary`. Это можно сделать с помощью закрытой проекции.

Закрытая проекция представляет собой интерфейс, который содержит геттеры для нужных нам полей. Чтобы Spring создал объект с полученными из базы данных значениями, нам нужно:

1. Присвоить имена геттерам такие же, как и геттеры в сущности (Entity).
2. Корректно указать возвращаемый тип у геттера.
3. Использовать проекцию в сигнатуре метода репозитория.

```
public interface EmployeeView {
    String getName();
    Integer getSalary();
}
```

```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {
    @Query(value = "SELECT * FROM employee",
        nativeQuery = true)
    List<EmployeeView> findAllEmployeeView();
}
```

Если мы добавим этот метод к контроллеру и вызовем его, то получим значения наших сотрудников (`Employee`), но без поля `id`, потому что это поле не указали в проекции.

Request URL

`http://localhost:8080/employees/view`

Server response

Code

Details

200

Response body

```
[
  {
    "name": "string",
    "salary": 0
  },
  {
    "name": "string",
    "salary": 0
  },
  {
    "name": "string",
    "salary": 1000
  },
  {
    "name": "B",
    "salary": 0
  },
  {
    "name": "string",
    "salary": 0
  }
]
```

Открытые проекции позволяют нам определять методы интерфейса с возвращаемыми значениями, вычисляемыми во время выполнения. Для этого нам нужно:

1. Создать в интерфейсе метод с именем, отличным от геттеров сущности (Entity).
2. Указать над этим методом аннотацию `@Value` с логикой вычисления.
3. Использовать проекцию в сигнатуре метода репозитория.

```
public interface EmployeeInfo {

    @Value("#{target.name + ' ' + target.salary}")
    String getFullInfo();

}
```

Использование проекции в репозитории:

```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {

    @Query(value = "SELECT * FROM employee",
        nativeQuery = true)
    List<EmployeeInfo> findAllEmployeeInfo();

}
```

В результате мы получаем такой ответ:

Request URL

http://localhost:8080/employees/info

Server response

Code	Details
200	Response body <pre>[{ "fullInfo": "string 0" }, { "fullInfo": "string 0" }, { "fullInfo": "string 1000" }, { "fullInfo": "B 0" }, { "fullInfo": "string 0" }]</pre>

Проекция на классе

Проекция на основе класса — **Class based projections** — позволяют нам работать с проекцией как с обычным классом. Чтобы этот класс работал с интерфейсом репозитория, имена параметров его конструктора должны совпадать со свойствами корневого класса сущностей.

Для этого надо провести процесс объединения сущностей. Создадим еще один класс-сущность `Position`, который будет содержать в себе информацию о должности сотрудника:

```
@Entity
@Table(name = "position")
public class Position {
    // Идентификатор должности, генерируется автоматически
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    // Название должности
    private String name;
    // Геттеры, сеттеры, конструкторы
}
```

И создадим связь one-to-one между `Position` и `Employee`.

```
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;
    private int salary;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "position_id")
    private Position position;

    // Геттеры, сеттеры, конструкторы
}
```

Предположим, нам нужно вернуть в контроллер информацию о сотруднике вместе с его должностью. Мы не можем использовать закрытую проекцию, т. к. у нас будет конфликт имен: в обеих сущностях есть поля `name`.

 Здесь мы можем воспользоваться выражением-конструктором: thorben-janssen.com/jpql/.

Создадим класс-проекцию:

```
public class EmployeeFullInfo {

    // Имя сотрудника
    private String name;

    // Зарплата сотрудника
    private Integer salary;

    // Название должности сотрудника
    private String positionName;

    // Конструктор класса EmployeeFullInfo
    public EmployeeFullInfo(String name, Integer salary, String positionName) {
        this.name = name;
        this.salary = salary;
        this.positionName = positionName;
    }

    // Геттеры и сеттеры
}
```

Важно создать в классе конструктор, принимающий на вход все интересующие нас параметры.

Далее в репозитории мы реализуем запрос.

```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {

    @Query("SELECT new ru.skypro.lessons.spring.model.projections." +
        "EmployeeFullInfo(e.name , e.salary , p.name) " +
        "FROM Employee e join fetch Position p " +
        "WHERE e.position = p")
    List<EmployeeFullInfo> findAllEmployeeFullInfo();

}
```

В запросе мы используем аннотацию `@Query`, а внутрь нее передаем JPQL-запрос.

В нем мы явно вызываем конструктор нашей проекции `EmployeeFullInfo` и передаем ему на вход поля Entity-классов.

В результате мы получаем значения в единой структуре:

Request URL

```
http://localhost:8080/employees/fullInfo
```

Server response

Code	Details
200	Response body <pre>[{ "name": "Вася", "salary": 125000, "positionName": "Аналитик" }, { "name": "Дарья", "salary": 250000, "positionName": "Разработчик" }, { "name": "Борис", "salary": 135000, "positionName": "Аналитик" }, { "name": "Виктория", "salary": 270000, "positionName": "Разработчик" }, { "name": "Иван", "salary": 15000, "positionName": "Тестировщик" }]</pre>

ORDER BY, LIMIT, MAX, MIN

Набор операторов нативного SQL помогут нам реализовывать запросы аналитического характера.

⚠ Операторы **MAX** и **MIN** являются агрегирующими операторами, также как **COUNT**, **SUM**, **AVG**. Если их применять без группировки, агрегация будет идти по всей таблице, а мы будем получать только одну строку с результатом.

```
select MIN(salary) "Минимальная зарплата", MAX(salary) "Максимальная зарплата"
from employee
```

Результат 1 ×

Таблица	123 Минимальная зарплата	123 Максимальная зарплата
1	15 000	270 000

Агрегация часто используется вкупе с группировкой по какому-либо признаку. Иногда необходимо узнать какой-либо показатель для каждой группы. Например, ниже представлены минимальные и максимальные зарплаты по должностям (**position_id**).

```
select MIN(salary) "Минимальная зарплата", MAX(salary) "Максимальная зарплата"
from employee
group by position_id
```

Результат 1 ×

123 Минимальная зарплата	123 Максимальная зарплата
15 000	15 000
125 000	135 000
250 000	270 000

Оператор ORDER BY

⚠ Оператор **ORDER BY** сортирует полученный результат запроса по набору полей, указанных после оператора.

На примере ниже — полный список сотрудников, отсортированный по зарплате:

```
select *
from employee e
order by salary
```

id	name	salary	position_id
5	Иван	15 000	3
1	Вася	125 000	2
3	Борис	135 000	2
2	Дарья	250 000	1
4	Виктория	270 000	1

По умолчанию сортировка происходит от меньшего к большему (ascending — восходящий).

Можно «развернуть» сортировку, указав ключевое слово DESC (descending — нисходящий).

```
select *
from employee e
order by salary desc
```

id	name	salary	position_id
4	Виктория	270 000	1
2	Дарья	250 000	1
3	Борис	135 000	2
1	Вася	125 000	2
5	Иван	15 000	3

Теперь список начинается с самого высокооплачиваемого сотрудника.

⚠ В оператор **ORDER BY** можно передавать несколько полей. Сравнение будет идти поочередно: если у строк первое значение будет одинаково, то сортировка будет происходить по второму значению и т. д.

```
select *
from employee e
order by position_id , salary
```

id	name	salary	position_id
2	Дарья	250 000	1
4	Виктория	270 000	1
1	Вася	125 000	2
3	Борис	135 000	2
5	Иван	15 000	3

В примере выше мы упорядочили таблицу по **id** должности, а затем по зарплате по восходящей.

Оператор LIMIT

⚠ Оператор **LIMIT** ограничивает число результирующих строк.

На примере ниже мы применяем оператор **LIMIT** с числом 3, в результате чего наш запрос вернет не больше 3 строк. Сейчас в таблице 5 строк, но благодаря оператору нам вернулось только 3.

```
select * , (select COUNT (*) from employee ) "Кол-во сотрудников"
from employee
limit 3
```

id	name	salary	position_id	Кол-во сотрудников
1	Вася	125 000	2	5
2	Дарья	250 000	1	5
3	Борис	135 000	2	5

Если мы передадим в **limit** значение больше, чем число результирующих строк, то это никак не скажется на полученной выборке.

```
select * , (select COUNT (*) from employee ) "Кол-во сотрудников"
from employee
limit 10
```

id	name	salary	position_id	Кол-во сотрудников
1	Вася	125 000	2	5
2	Дарья	250 000	1	5
3	Борис	135 000	2	5
4	Виктория	270 000	1	5
5	Иван	15 000	3	5

Очень важно понимать **порядок выполнения SQL-запроса**:



Исходя из этой схемы мы видим, что **ORDER BY** и **LIMIT** выполняются в самом конце сканирования таблиц и вычисления результатов.

Корректный SELECT

Предположим, нам необходимо найти сотрудника с самой высокой зарплатой. Зная операторы

`ORDER BY` и `LIMIT`, мы можем написать подобный запрос:

```
select *
from employee e
order by salary desc
limit 1
```

oyee 1 ×

Введите SQL выражение чтобы отфильтровать результаты

id	name	salary	position_id
4	Виктория	270 000	1

Запрос будет корректным, но:

- Он не будет оптимальным, потому что сортировка — дорогая операция.
- Подобный подход мы не сможем использовать в других случаях. Например, когда нам нужно найти сотрудника со средней зарплатой по всей фирме.
- Если у нас несколько сотрудников с максимальной зарплатой, запрос выше может вернуть нам лишь одного.

Поэтому лучше использовать **подзапрос**:

```
select *
from employee e
where salary = (select MAX(salary) from employee e2 )
```

mployee 1 ×

Введите SQL выражение чтобы отфильтровать результаты

id	name	salary	position_id
4	Виктория	270 000	1

В запросе выше мы находим всех сотрудников, чья зарплата равна максимальной зарплате по фирме. Используя такую логику, мы:

- увеличиваем скорость выполнения запроса;
- получаем нужное количество сотрудников, которые удовлетворяют нашим критериям.